

---

# DART: Demand-Addressed Runtime Tokens for Interest-Driven Decode

---

ARXIV PREPRINT

Aditya Raj  
Independent Researcher

September 2026

## ABSTRACT

Autoregressive LLM serving still treats decode as a push problem: the engine generates as fast as the GPU allows, then a byte pipe tries to deliver. A token with no consumer credit is a wasted residual-stream step—energy, KV pages, and batch slots spent encoding a segment nobody asked to absorb. Andes noticed that humans read slower than GPUs generate, then still generated and buffered. This paper inverts the stack. *Interest-Driven Decode* (IDD) makes an outstanding Interest the only thing that may run a decode kernel. DART (Demand-Addressed Runtime Tokens) is the continuation runtime that implements IDD: named objects addressed by `/cip/<model-hash>/<kv-root>/tokens/seg/<i>`, HMAC leases, AIMD congestion control over a token window  $W$ , a content-addressed store, pinned KV by Merkle `kv_root`, and Interest routing (CAS, then pin holder, then cheapest node plus handover). The kernel remains vLLM, llama.cpp, or an in-process Hugging Face model. On the in-repository paper suite, a 30 tok/s reading pacer cuts generated tokens 52% and KV high-water 43% versus push, Andes captures only 68% of that win and still holds unread inventory, grammar jump-forward is one kernel versus 36 under logit masking, a FileCAS peer answers with zero engine launches, and an in-process waiting plugin admits once while HTTP adapters re-enter admission. DART is a control plane, not a GPU fleet.

**Keywords.** LLM serving, interest-driven decode, named continuations, congestion control, KV cache

## 1 Introduction

Today’s serving stacks maximize occupancy [1–3]. The scheduler packs sequences. The network is a pipe. The client is a sink. Prefill/decode disaggregation [4, 5] and KV-centric fabrics [6, 7] move the residual stream; they do not ask whether it should run.

That question matters. Autoregression is the expensive encoder. If the compositor, TTS mouth, or JSON parser cannot absorb the next segment, encoding it is waste. Andes [8] couples a watermark pause to generation so a human reader is not flooded. It still generates into a buffer. The inventory of unread tokens is the tell.

**Interest-Driven Decode** inverts the default:

Outstanding Interests are the only thing that may run a decode kernel.

Zero Interests means zero decode. A cache hit never touches the kernel. Changing machines is not live-migration of an HTTP request; it is a different producer answering the next named Interest.

**DART** (Demand-Addressed Runtime Tokens) is the runtime that makes that invariant true in front of an existing engine. After prefill it publishes a capability—model fingerprint, Merkle `kv_root`, anti-amplification lease—not a process snapshot. Consumers issue Interests for a token window  $W$ . The scheduler *is* the congestion controller. Named Data is immutable and cacheable; a second process with FileCAS can answer without a GPU.

This paper is the system that is implemented, not a fleet we did not build. HTTP adapters to vLLM and llama.cpp already sit on the same CIP. They re-enter admission. An in-process waiting plugin parks the request in waiting with pinned blocks when  $W=0$ . Evaluation uses the in-repo paper suite (`dart experiment -suite paper`) on a synthetic producer with real KV-extent accounting, plus mesh, CAS-peer, grammar, and waiting-plugin ablations.

**Contributions.** (1) IDD as a primitive: credit-gated decode on named continuations. (2) CIP, leases, and AIMD over tokens, not bytes. (3) CAS  $\rightarrow$  pin  $\rightarrow$  cheapest-plus-handover routing without re-prefill. (4) An Andes-complete kill test and honest limits of the HTTP path.

## 2 Related work

**Occupancy serving.** Orca [2] and vLLM [1] made continuous batching and paged KV the default. Sarathi-Serve [3] splits chunks to hide prefill bubbles. FlashAttention [9] and TPU serving [10] made the kernel cheaper; they did not change *whether* it may run.

**Disaggregation and KV movement.** DistServe [4] and Splitwise [5] split prefill from decode. Mooncake [6], CacheGen/LMCache [7], Llumnix [11], and NIXL-style RDMA move KV between machines. DART reuses those movers as connectors. Handover is triggered by an Interest with a new locator, not by a control plane rescheduling a request.

**QoE pause.** Andes [8] pauses generation against a display watermark. If that pause captured  $\geq 90\%$  of IDD’s generated-token win versus push, CIP would be an Andes patch (kill criterion 1). It does not: Andes still holds inventory; IDD does not generate it.

**Named data and media.** NDN [12] named immutable objects; an Interest was not a GPU capability. MOQT [13] maps live media onto QUIC objects. A future profile can map CIP Data onto MOQ; it must not become the decode authorizer. SGLang [14] and XGrammar [15] jump structured spans inside one process. DART names the span so a peer can cache it.

## 3 Interest-Driven Decode

A live generation is an address space, not a socket.

$$/cip/\langle h_{\text{model}} \rangle / \langle kv\_root \rangle / \text{tokens} / \text{seg} / \langle i \rangle \quad (1)$$

Immutable Data is keyed by  $(h_{\text{model}}, kv\_root, kind, i)$  and never overwritten. The  $kv\_root$  in the name is the prefix *before* the segment; the body carries the child root. The mutable cursor is a signed `ContinuationLease`: window cap, decode quota, expiry, startup credit.

An Interest is a compute capability: name, token window  $W$ , lifetime  $T$ , locator, HMAC lease. On a CAS hit the producer returns Data and does not call the engine. Otherwise it grants  $\min(W, cwnd)$  credits and may decode at most that many tokens. On lifetime expiry it stops, pins KV, and discards speculative draft. A Nack names the refusal (`no_credit`, `no_model`, `busy`, `expired`, `amplification`, `done`).

Disconnect (SSE cancel, WebSocket close, DELETE) zeroes credits. That is how backpressure reaches the kernel, which stock vLLM SSE does not.

## 4 DART

### 4.1 Control loop

Figure 1 is the implemented loop. Clients (compositor, TTS, JSON parser, tool runtime) estimate a receive window and emit Interests. Pacers in the SDK: `DrainPacer` (API pull), `ReadingPacer` (default 30 tok/s), `TtsPacer`, `JsonNeedPacer`.

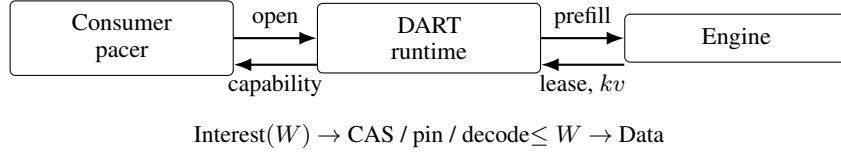


Figure 1: Prefill once. Subsequent work is credited Interests against a named continuation.

Table 1: Push drain versus IDD at 30 tok/s. Unconsumed tokens are generated minus consumed. Survive: generated  $< 0.9 \times$  push or lower KV high-water, with consumption  $> 0$ .

| Arm          | Gen. | Cons. | Uncons. | Eng. launches | KV high-water |
|--------------|------|-------|---------|---------------|---------------|
| Push (drain) | 96   | 96    | 0       | 6             | 1.84 MB       |
| IDD 30 tok/s | 46   | 46    | 0       | 31            | 1.05 MB       |

## 4.2 Congestion control is the scheduler

State per continuation:  $cwnd$ , credits, in-flight,  $ssthresh$ , RTT EWMA. Invariant:  $credits + in\text{-}flight \leq \min(cwnd, W_{\max}^{lease}, W_{\max})$ . Slow-start then AIMD on ACK (the next Interest). Timeout halves the window. Speculative  $K = \text{clamp}(0, \text{round}(cwnd \cdot RTT / t_{\text{decode}} - cwnd), K_{\max})$  is exported as a TCP-style prefetch *signal*. v1 does not run uncredited residual-stream steps: that would violate IDD, and HTTP adapters cannot roll back a draft.

Credit-ready continuations pack up to `max_num_seqs`.  $W=0$  does not enter the batch. If the ready set is too small, DART does not mint unread tokens to fill the GPU; a cluster should fill with other paid work (prefill, embeddings, offline batch).

## 4.3 Pin, connector, mesh

PinnedKVPool indexes engine state by `kv_root`. Sleep does not drop identity; expiry does. adopt installs named KV and skips prefill. Connectors expose `put/get/transfer`: memory, file, LMCache-shaped, NIXL-shaped. Without those libraries, transfer is memcpy and metrics report `rdma_available=false`.

InterestRouter order: (1) CAS already has the name; (2) live pin holder, even if expensive; (3) live continuation; (4) cheapest node, NIXL-shaped pull, adopt, decode. `dart mesh -nodes 3` is the local form.

## 4.4 Engines

DART never owns the residual stream. SyntheticEngine is the kill-test producer (hash-based tokens, real layer $\times$ block KV extents). VLLMChatEngine and LlamaCppEngine turn each credited Interest into  $\text{max\_tokens} = W / n_{\text{predict}} = W$ . HuggingFaceEngine runs in-process weights (demo: SmolLM2-135M-Instruct). InProcessVLLMEngine admits once, then parks in waiting with pinned blocks when  $W=0$ . CacheOnlyEngine never decodes.

OpenAI clients keep working: `POST /v1/chat/completions` with X-Dart-Pace / X-Dart-Window. Without pace, the facade uses drain credit as SSE is pulled—already stronger than stock vLLM, which keeps the kernel running after the gateway stops reading.

## 5 Evaluation

All numbers in this section are from `dart experiment -suite paper (-seconds 1.0 -max-tokens 96)` on the synthetic producer unless noted. Paper tables report *engine* kernel launches (successful generate/completion), not only wrapper ticks.

### 5.1 Kill test: push versus credit

Table 1 is the inversion test. IDD generates 52% fewer tokens and cuts KV high-water 43%. It launches *more* kernels (31 vs. 6) because each Interest is a short  $W$ , not one long push. The engine still only runs when credited; launch count is not the objective. The kill test survives.

Table 2: Andes watermark pause versus IDD refuse-to-decode. Capture is Andes’ generated-token win over push divided by IDD’s. Kill if capture  $\geq 90\%$  and Andes inventory stays at the watermark.

| Arm              | Generated | Eng. launches | Inventory HW | Capture of IDD |
|------------------|-----------|---------------|--------------|----------------|
| Push             | 96        | 6             | —            | —              |
| Andes ( $W=32$ ) | 62        | 4             | 32           | 0.68           |
| IDD 30 tok/s     | 46        | 31            | 0            | 1.00           |

## 5.2 Andes-complete

Table 2: Andes captures 68% of IDD’s win and holds 32 unread tokens. IDD inventory high-water is 0. The Andes-complete kill does *not* fire. CIP is not an Andes patch.

## 5.3 Grammar, CAS peer, mesh, waiting

Grammar jump-forward emits one named Data in one kernel. Logit masking pays one kernel per constrained token: 36 launches versus 1 (ratio 36) on the `tool-args` span.

A producer writes FileCAS; a second process with `CacheOnlyEngine` satisfies the same CIP name with `cache_hit=true` and 0 engine launches. That is the not-Andes defense: the continuation object is the cache unit.

Three-node in-process mesh with a NIXL-shaped connector: first Interest pins on node-0; repeat is CAS (kernel vector unchanged); next segment stays on the pin holder even though node-2 is cheaper; after `release_for_handover`, node-2 adopts without a new prefill (`prefills_skipped`) and decodes.

Waiting plugin versus HTTP fake-vLLM: idle  $W=0$  sits in `waiting` with 8 pinned blocks, 1 admission, 0 decode forwards. Two Interests: 2 decode forwards, still 1 admission, 0 re-entries. Pinned blocks survive memory pressure that evicts an unpinned victim. The HTTP path issues 2 generate POSTs (admission twice).

## 6 Limitations

DART is not a finished GPU fleet that replaces vLLM. HTTP `VLLMChatEngine` / `LlamaCppEngine` re-enter admission; a load spike can 503 a live continuation; prefix cache may evict and force re-prefill. IDD still did not decode without credit; it may pay a re-prefill pinned blocks would have avoided. Measure `GET /v1/engine`: local POSTs versus the engine process forward counter.

Real NIXL RDMA and LMCACHE GPU pages are optional. Tests use `memcpy`. `CreditGatedScheduler` binds to a live GPU vLLM process only when the `vllm` package is present and `DART_VLLM_INPROCESS=1`.

If credit gating dropped consumed tok/J because the GPU went idle, fill-from-other-demand must be enabled before declaring death (kill criterion 2). Per-token names are forbidden; segments stay 8–32 tokens (kill criterion 3).

## 7 Conclusion

Serving should not encode segments nobody has credit to consume. DART makes that a protocol: named continuations, token windows, CAS then pin then decode, handover as the next Interest from a new locator. The kernels stay the kernels. The scheduler is a congestion controller. The paper suite is in the repository; the Andes-complete kill did not fire; the HTTP honesty clause remains.

## References

- [1] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.
- [2] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation*, 2022.

- [3] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation*, 2024.
- [4] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation*, 2024.
- [5] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st Annual International Symposium on Computer Architecture*, 2024.
- [6] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. *arXiv preprint arXiv:2407.00079*, 2024.
- [7] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024.
- [8] Jiachen Liu, Zhiyu Lu, Shuowei Shi, Alexander Putta, Peter Wang, Lymeng Chen, Ang Chen, Fan Zhang, Yiping Cheng, Yifan Du, et al. Andes: Defining and Enhancing Quality-of-Experience in LLM-Based Text Streaming Services. *arXiv preprint arXiv:2404.16283*, 2024.
- [9] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*, 2022.
- [10] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently Scaling Transformer Inference. In *Proceedings of Machine Learning and Systems*, 2023.
- [11] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic Scheduling for Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation*, 2024.
- [12] Van Jacobson, Diana K. Smetters, James D. Thornton, Michael F. Plass, Nicholas H. Briggs, and Rebecca L. Braynard. Networking Named Content. In *Proceedings of the 5th International Conference on Emerging Networking Experiments and Technologies*, 2009.
- [13] Luke Curley, Kirill Pugin, Suhas Nandakumar, Victor Vasiliev, and Ian Swett. Media over QUIC Transport. Internet-Draft draft-ietf-moq-transport, 2024.
- [14] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient Execution of Structured Language Model Programs. *arXiv preprint arXiv:2312.07104*, 2024.
- [15] Yixin Dong, Charlie F. Jiang, Yilong Yang, Yuzhen Lin, Zixuan Zhu, Chen Li, et al. XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models. *arXiv preprint arXiv:2411.15100*, 2024.